

Corrección base en apiClient.js

Antes de implementar las recomendaciones, se identificó y corrigió un bug en la configuración base del proyecto. El archivo apiClient.js exportaba únicamente la instancia de Axios, pero blueprintsService.js esperaba un objeto con los métodos getAll, getByAuthor, getByAuthorAndName y create. Esto causaba que todas las llamadas al servicio fallaran en tiempo de ejecución.

```
const apiclient = {
  getAll: async () => {
    const { data } = await api.get('/blueprints')
    return data
  },
  getByAuthor: async (author) => {
    const { data } = await api.get(`/blueprints/${author}`)
    return data
  },
  getByAuthorAndName: async (author, name) => {
    const { data } = await api.get(`/blueprints/${author}/${name}`)
    return data
  },
  create: async (blueprint) => {
    const { data } = await api.post('/blueprints', blueprint)
    return data
  },
}

export { api }
export default apiclient
```

1. Redux Avanzado

1.1 Estados loading/error por thunk

El problema encontrado era que solo el thunk `fetchAuthors` tenía los tres casos necesarios `pending`, `fulfilled` y `rejected`. Los demás thunks como `fetchByAuthor`, `fetchBlueprint` y `createBlueprint` únicamente manejaban el caso `fulfilled`, por lo que cuando fallaban, el estado `status` nunca cambiaba a `failed` y el campo `error` nunca se actualizaba.

```
.addCase(fetchByAuthor.fulfilled, (s, a) => {
  s.byAuthor[a.payload.author] = a.payload.items
})
.addCase(fetchBlueprint.fulfilled, (s, a) => {
  s.current = a.payload
})
```

La corrección consistió en agregar los casos `pending` y `rejected` a todos los thunks en `blueprintsSlice.js`:

```
reducers: {},
extraReducers: (builder) => {
  builder
    // fetchAuthors
    .addCase(fetchAuthors.pending, (s) => {
      s.status = 'loading'
      s.error = null
    })
    .addCase(fetchAuthors.fulfilled, (s, a) => {
      s.status = 'succeeded'
      s.authors = a.payload
    })
    .addCase(fetchAuthors.rejected, (s, a) => {
      s.status = 'failed'
      s.error = a.error.message
    })
    // fetchByAuthor - los 3 casos
    .addCase(fetchByAuthor.pending, (s) => {
      s.status = 'loading'
      s.error = null
    })
    .addCase(fetchByAuthor.fulfilled, (s, a) => {
      s.status = 'succeeded'
      s.byAuthor[a.payload.author] = a.payload.items
    })
    .addCase(fetchByAuthor.rejected, (s, a) => {
      s.status = 'failed'
      s.error = a.error.message
    })
    // fetchBlueprint
    .addCase(fetchBlueprint.pending, (s) => { s.status = 'loading' })
    .addCase(fetchBlueprint.fulfilled, (s, a) => {
      s.status = 'succeeded'
      s.current = a.payload
    })
    .addCase(fetchBlueprint.rejected, (s, a) => {
      s.status = 'failed'
      s.error = a.error.message
    })
  },
})

export default slice.reducer
```

También se actualizó BlueprintsPage.jsx para mostrar esos estados en la interfaz de usuario:

```
<div className="card">
  <h3 style={{ marginTop: 0 }}>
    {selectedAuthor ? `${selectedAuthor}'s blueprints:` : 'Results'}
  </h3>
  {status === 'loading' && <p>Cargando...</p>}
  {status === 'failed' && [
    <div style={{ color: '#f87171' }}>
      ⚠️ {error}
      <button onClick={() => dispatch(fetchByAuthor(selectedAuthor))}>
        Reintentar
      </button>
    </div>
  ]}
  {!items.length && status !== 'loading' && <p>Sin resultados.</p>}
  {!!items.length && (
    <div style={{ overflowX: 'auto' }}>
      <table style={{ width: '100%', borderCollapse: 'collapse' }}>
        <thead>
          <tr>
            <th
              style={{
                textAlign: 'left',
                padding: '8px',
                borderBottom: '1px solid #334155',
              }}
            >
              Blueprint name
            </th>
            <th
              style={{
```

1.2 Memo selectors: Top 5 blueprints

Se implementó un selector memoizado con createSelector para derivar los 5 blueprints con más puntos del autor actualmente seleccionado, evitando recálculos innecesarios cuando el estado no cambia:

```
features > blueprints > JS blueprintsSlice.js > slice > extraReducers
const slice = createSlice({
  extraReducers: (builder) => {
    //
    .addCase(fetchByAuthor.rejected, (s, a) => {
      s.status = 'failed'
      s.error = a.error.message
    })
  },
})

export default slice.reducer

import { createSelector } from '@reduxjs/toolkit'

export const selectTop5 = createSelector(
  (state) => state.blueprints.byAuthor,
  (_, author) => author,
  (byAuthor, author) => {
    const list = byAuthor[author] ?? []
    return [...list]
      .sort((a, b) => (b.points?.length ?? 0) - (a.points?.length ?? 0))
      .slice(0, 5)
  }
)
```

2. Rutas Protegidas

Para proteger rutas que requieren autenticación se implementaron los siguientes cambios en orden:

2.1 authSlice y store

Se creó authSlice.js con las acciones loginSuccess y logout, y se registró el reducer en el store:

```
1  import { createSlice } from '@reduxjs/toolkit'
2
3  const authSlice = createSlice({
4    name: 'auth',
5    initialState: {
6      token: localStorage.getItem('token') ?? null,
7      isAuthenticated: !!localStorage.getItem('token'),
8    },
9    reducers: {
10     loginSuccess(state, action) {
11       state.token = action.payload
12       state.isAuthenticated = true
13       localStorage.setItem('token', action.payload)
14     },
15     logout(state) {
16       state.token = null
17       state.isAuthenticated = false
18       localStorage.removeItem('token')
19     },
20   },
21 })
22
23 export const { loginSuccess, logout } = authSlice.actions
24 export default authSlice.reducer
```

```
1  import { configureStore } from '@reduxjs/toolkit'
2  import blueprintsReducer from '../features/blueprints/blueprintsSlice.js'
3  import authReducer from '../features/auth/authSlice'
4
5  const store = configureStore({
6    reducer: {
7      blueprints: blueprintsReducer,
8      auth: authReducer,
9    },
10  })
11
12 export default store
13
```

2.2 Componente PrivateRoute

Se creó el componente PrivateRoute.jsx que lee el estado de autenticación desde Redux y redirige al login si el usuario no está autenticado:

```
import { useSelector } from 'react-redux'
import { Navigate } from 'react-router-dom'

export default function PrivateRoute({ children }) {
  const isAuthenticated = useSelector((state) => state.auth.isAuthenticated)
  return isAuthenticated ? children : <Navigate to="/login" replace />
}
```

2.3 LoginPage con dispatch

Se modificó LoginPage.jsx para usar dispatch(loginSuccess(token)) en lugar de escribir directamente en localStorage, manteniendo así Redux como fuente de verdad:

```
import { useState } from 'react'
import api from '../services/apiClient.js'
import { useDispatch } from 'react-redux'
import { loginSuccess } from '../features/auth/authSlice'

export default function LoginPage() {
  const [username, setUsername] = useState('')
  const [password, setPassword] = useState('')
  const [error, setError] = useState(null)
  const dispatch = useDispatch()

  const submit = async (e) => {
    e.preventDefault()
    setError(null)
    try {
      const { data } = await api.post('/auth/login', { username, password })

      const tokenRecibido = data.access_token || data.token;

      if (tokenRecibido) {
        dispatch(loginSuccess(tokenRecibido))
        alert('Login exitoso');
      } else {
        console.error("El servidor no envió 'access_token' ni 'token'", data);
        setError('Error en el formato del token');
      }
    } catch (e) {
      setError('Credenciales inválidas o servidor no disponible');
    }
  }
}
```

2.4 Protección de rutas en App.jsx

Finalmente, la ruta /create se envolvió en el componente PrivateRoute para que solo usuarios autenticados puedan acceder al formulario de creación:

```
import { NavLink, Route, Routes } from 'react-router-dom'
import BlueprintsPage from './pages/BlueprintsPage.jsx'
import BlueprintDetailPage from './pages/BlueprintDetailPage.jsx'
import LoginPage from './pages/LoginPage.jsx'
import NotFound from './pages/NotFound.jsx'
import PrivateRoute from './components/PrivateRoute.jsx'
import BlueprintForm from './components/BlueprintForm.jsx'

export default function App() {
  return (
    <div className="container">
      <header>
        <h1>ECI - Laboratorio de Blueprints en React</h1>
        <nav>
          <NavLink to="/" end>Blueprints</NavLink>
          <NavLink to="/login">Login</NavLink>
        </nav>
      </header>
      <Routes>
        <Route path="/" element={<BlueprintsPage />} />
        <Route path="/blueprints/:author/:name" element={<BlueprintDetailPage />} />
        <Route path="/login" element={<LoginPage />} />
        <Route
          path="/create"
          element={
            <PrivateRoute>
              <BlueprintForm onSubmit={() => {}} />
            </PrivateRoute>
          }
        />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </div>
  )
}
```

3. CRUD Completo

3.1 Métodos update y remove

Se agregaron los métodos update y remove al objeto apiClient para soportar las operaciones PUT y DELETE contra el backend:

```
const apiClient = {
  getAll: async () => {
    const { data } = await api.get('/blueprints')
    return data
  },
  getByAuthor: async (author) => {
    const { data } = await api.get(`/blueprints/${author}`)
    return data
  },
  getByAuthorAndName: async (author, name) => {
    const { data } = await api.get(`/blueprints/${author}/${name}`)
    return data
  },
  create: async (blueprint) => {
    const { data } = await api.post('/blueprints', blueprint)
    return data
  },
  update: async (author, name, blueprint) => {
    const { data } = await api.put(`/blueprints/${author}/${name}`, blueprint)
    return data
  },
  remove: async (author, name) => {
    await api.delete(`/blueprints/${author}/${name}`)
  },
}
```

Los mismos métodos se implementaron en apimock.js para mantener la interfaz consistente durante el desarrollo:

```
const apimock = {
  getAll: async () => mockData,
  getByAuthor: async (author) => mockData.filter(bp => bp.author === author),
  getByAuthorAndName: async (author, name) =>
    mockData.find(bp => bp.author === author && bp.name === name),
  create: async (blueprint) => {
    mockData.push(blueprint);
    return blueprint;
  },
  update: async (author, name, blueprint) => {
    const index = mockData.findIndex(bp => bp.author === author && bp.name === name)
    if (index === -1) throw new Error('Blueprint not found')
    mockData[index] = { ...mockData[index], ...blueprint }
    return mockData[index]
  },
  remove: async (author, name) => {
    const index = mockData.findIndex(bp => bp.author === author && bp.name === name);
    if (index !== -1) {
      mockData.splice(index, 1);
    }
  }
};

export default apimock;
```

3.2 Thunk deleteBlueprint en el slice

```
.addCase(deleteBlueprint.fulfilled, (s, a) => {
  const { author, name } = a.payload
  if (s.byAuthor[author]) {
    s.byAuthor[author] = s.byAuthor[author].filter(bp => bp.name !== name)
  }
  if (s.current?.name === name) s.current = null
})
.addCase(deleteBlueprint.rejected, (s, a) => {
  s.status = 'failed'
  s.error = a.error.message
})
},
})
```

Se añadió el botón Delete en la tabla de BlueprintsPage.jsx:

```
export default function BlueprintsPage() {  
  // ...  
  </thead>  
  <tbody>  
    {items.map(bp => (  
      <tr key={bp.name}>  
        <td style={{ padding: '8px', borderBottom: '1px solid #1f2937' }}>  
          {bp.name}  
        </td>  
        <td style={{ padding: '8px', text-align: 'right', borderBottom: '1px solid #1f2937' }}>  
          {bp.points?.length || 0}  
        </td>  
        <td style={{ padding: '8px', borderBottom: '1px solid #1f2937' }}>  
          <button className="btn" onClick={() => openBlueprint(bp)}>  
            Open  
          </button>  
          <button className="btn" style={{ color: '#f87171' }} onClick={() => handleDelete(bp)}>  
            Delete  
          </button>  
        </td>  
      </tr>  
    )  
    )  
  </tbody>  
</table>  
</div>  
))  
<p style={{ marginTop: 12, fontWeight: 700 }}>Total user points: {totalPoints}</p>  
</div>  
</function>
```

3.3 Optimistic updates

El delete inicial era pesimista: Esperaba la confirmación del servidor antes de actualizar el estado local. Se modificó para ser optimista, eliminando el elemento del estado inmediatamente y revirtiendo el cambio si el servidor responde con error:

```
export const deleteBlueprint = createAsyncThunk(  
  'blueprints/deleteBlueprint',  
  async ({ author, name }, { dispatch, rejectWithValue, getState }) => {  
    const snapshot = getState().blueprints.byAuthor[author]?.find(bp => bp.name === name)  
    dispatch(removeBlueprintOptimistic({ author, name }))  
    try {  
      await blueprintsService.remove(author, name)  
      return { author, name }  
    } catch (err) {  
      dispatch(fetchByAuthor(author))  
      return rejectWithValue(err.message)  
    }  
  })  
)  
  
const slice = createSlice({  
  name: 'blueprints',  
  initialState: {  
    authors: [],  
    byAuthor: {},  
    current: null,  
    status: 'idle',  
    error: null,  
  },  
  reducers: {  
    removeBlueprintOptimistic(state, action) {  
      const { author, name } = action.payload  
      if (state.byAuthor[author]) {  
        state.byAuthor[author] = state.byAuthor[author].filter(bp => bp.name !== name)  
      }  
    },  
  },  
  extraReducers: (builder) => {  
    builder
```


3.4 Pruebas del CRUD

Se verificó el flujo completo de la tabla: Open dibuja el blueprint en el canvas, Delete lo elimina de la lista y el contador de puntos se actualiza correctamente.

ECI - Laboratorio de Blueprints en React

BlueprintsLogin

Blueprints

juan

Get blueprints

juan's blueprints:

Blueprint name	Number of points	Actions
tree	14	Open Delete

Total user points: 14

Current blueprint: tree

ECI - Laboratorio de Blueprints en React

BlueprintsLogin

Blueprints

juan

Get blueprints

juan's blueprints:

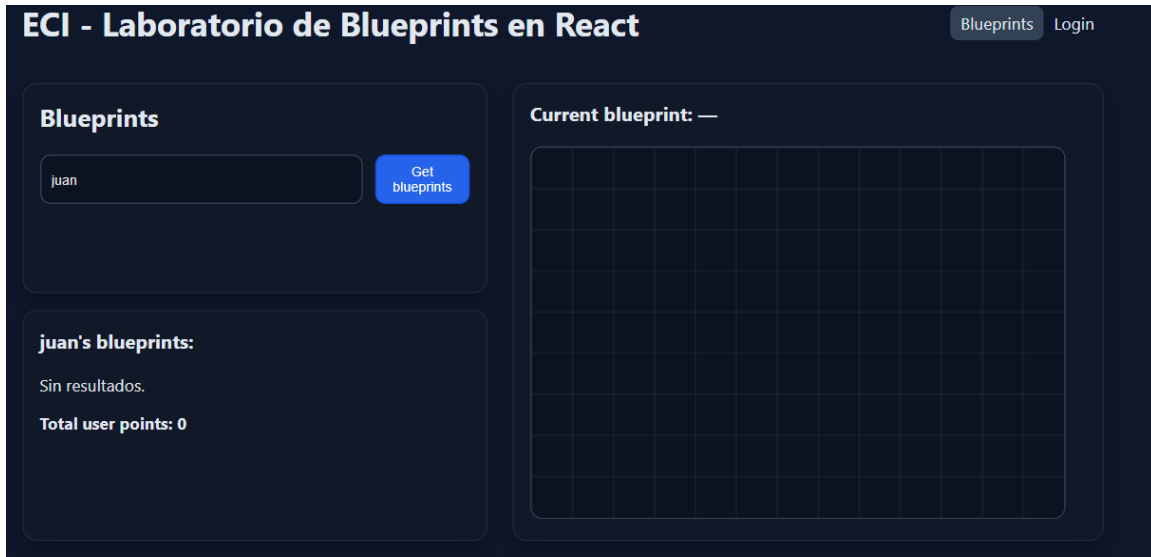
Blueprint name	Number of points	Actions
tree	14	Open Delete

Total user points: 14

Current blueprint: tree

localhost:5173 dice
¿Eliminar "tree"?

AceptarCancelar



4. Dibujo Interactivo

4.1 InteractiveCanvas.jsx

Se reemplazó el SVG estático por un lienzo interactivo donde el usuario puede hacer clic para agregar puntos. Cada clic registra las coordenadas con corrección de escala y dibuja el punto en el canvas:

```
import { useEffect, useRef } from 'react'

export default function InteractiveCanvas({ points = [], onPointAdded, width = 520, height = 360 }) {
  const ref = useRef(null)

  // Redibuja cada vez que cambian los puntos
  useEffect(() => {
    const canvas = ref.current
    if (!canvas) return
    const ctx = canvas.getContext('2d')

    ctx.clearRect(0, 0, canvas.width, canvas.height)
    ctx.fillStyle = '#0b1220'
    ctx.fillRect(0, 0, canvas.width, canvas.height)

    // Grilla
    ctx.strokeStyle = 'rgba(148,163,184,0.12)'
    ctx.lineWidth = 1
    for (let x = 0; x <= canvas.width; x += 40) {
      ctx.beginPath(); ctx.moveTo(x, 0); ctx.lineTo(x, canvas.height); ctx.stroke()
    }
    for (let y = 0; y <= canvas.height; y += 40) {
      ctx.beginPath(); ctx.moveTo(0, y); ctx.lineTo(canvas.width, y); ctx.stroke()
    }

    // Segmentos
    if (points.length > 1) {
      ctx.strokeStyle = '#93c5fd'
      ctx.lineWidth = 2
      ctx.beginPath()
      ctx.moveTo(points[0].x, points[0].y)
      for (let i = 1; i < points.length; i++) {
        ctx.lineTo(points[i].x, points[i].y)
      }
      ctx.stroke()
    }

    // Puntos
    ctx.fillStyle = '#fbbf24'
    for (const p of points) {
      ctx.beginPath()
      ctx.arc(p.x, p.y, 5, 0, Math.PI * 2)
      ctx.fill()
    }
  }, [points])

  return (
    <div style={width: width, height: height} ref={ref}>
      <img alt="Interactive canvas with grid and points" data-bbox="244 495 683 859"/>
    </div>
  )
}
```

```

const handleClick = (e) => {
  if (!onPointAdded) return
  const rect = ref.current.getBoundingClientRect()
  const scaleX = ref.current.width / rect.width
  const scaleY = ref.current.height / rect.height
  const x = Math.round((e.clientX - rect.left) * scaleX)
  const y = Math.round((e.clientY - rect.top) * scaleY)
  onPointAdded({ x, y })
}

return (
  <canvas
    ref={ref}
    width={width}
    height={height}
    onClick={handleClick}
    style={{
      background: '□ #0b1220',
      border: '1px solid □ #334155',
      borderRadius: 12,
      cursor: 'crosshair',
      width: '100%',
      maxWidth: width,
      display: 'block',
    }}
  />
)

```

4.2 Thunk updateBlueprint

Se agregó el thunk updateBlueprint en blueprintsSlice.js para enviar los cambios al backend mediante PUT:

```

    .addCase(updateBlueprint.fulfilled, (s, a) => {
      const { author, name, points } = a.payload
      if (s.current?.name === name) {
        s.current = { ...s.current, points }
      }
      if (s.byAuthor[author]) {
        const idx = s.byAuthor[author].findIndex(bp => bp.name === name)
        if (idx !== -1) s.byAuthor[author][idx] = { ...s.byAuthor[author][idx], points }
      }
    })
    .addCase(updateBlueprint.rejected, (s, a) => {
      s.status = 'failed'
      s.error = a.payload
    })
  },
)

```

4.3 BlueprintDetailPage con botón Guardar

La lógica central es que el array de puntos vive en el estado local del componente useState, no en Redux, porque representa una edición en progreso. Solo al hacer clic en Guardar se despacha updateBlueprint y el estado de Redux se actualiza:

```
useEffect(() => {
  dispatch(fetchBlueprint({ author, name }))
}, [author, name, dispatch])

useEffect(() => {
  if (bp) setPoints(bp.points ?? [])
}, [bp])

const handlePointAdded = (newPoint) => {
  setPoints((prev) => [...prev, newPoint])
}

const handleSave = () => {
  dispatch(updateBlueprint({ author: bp.author, name: bp.name, points }))
  alert('Blueprint guardado')
}

const handleClear = () => setPoints([])

if (!bp) return <div className="card"><p>Cargando...</p></div>

return (
  <div className="card">
    <h2 style={{ marginTop: 0 }}>{bp.name}</h2>
    <p><strong>Autor:</strong> {bp.author}</p>
    <p><strong>Puntos:</strong> {points.length}</p>

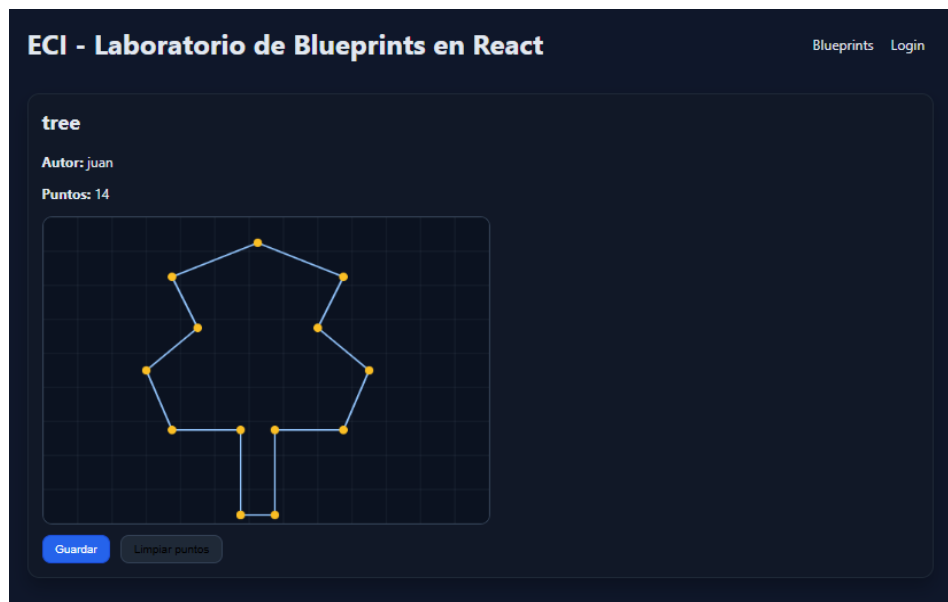
    <InteractiveCanvas
      points={points}
      onPointAdded={handlePointAdded}
    />

    <div style={{ display: 'flex', gap: 12, marginTop: 12 }}>
      <button
        className="btn primary"
        onClick={handleSave}
        disabled={status === 'loading'}>
        Guardar
      </button>
      <button
        className="btn"
        onClick={handleClear}>
        Limpiar puntos
      </button>
    </div>

    {error && <p style={{ color: '#f87171', marginTop: 8 }}>▲ {error}</p>}
  </div>
)
```

4.4 Evidencia

- Accediendo a <http://localhost:5173/blueprints/juan/tree> se visualiza el árbol original:



- Al hacer clic en el canvas se agrega un punto amarillo y el contador se incrementa:



- Al guardar, el blueprint actualizado se envía al servidor:



- El botón Limpiar puntos elimina todos los puntos del canvas:



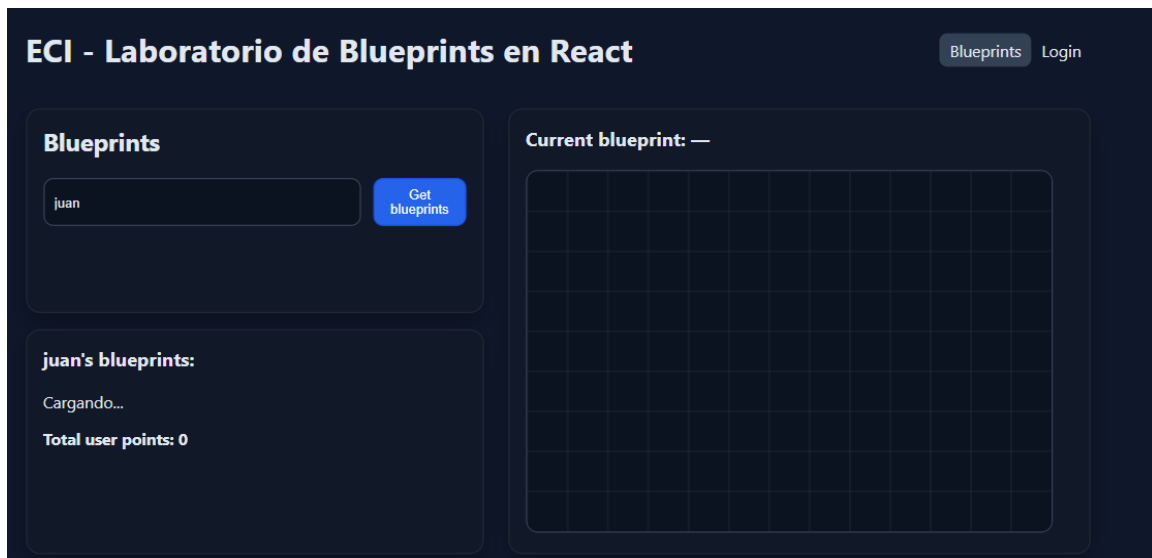
5. Errores y Retry

Esta funcionalidad ya estaba implementada desde el punto 1. Cuando un GET falla, se muestra un banner de error y un botón "Reintentar" que vuelve a despachar el thunk correspondiente:

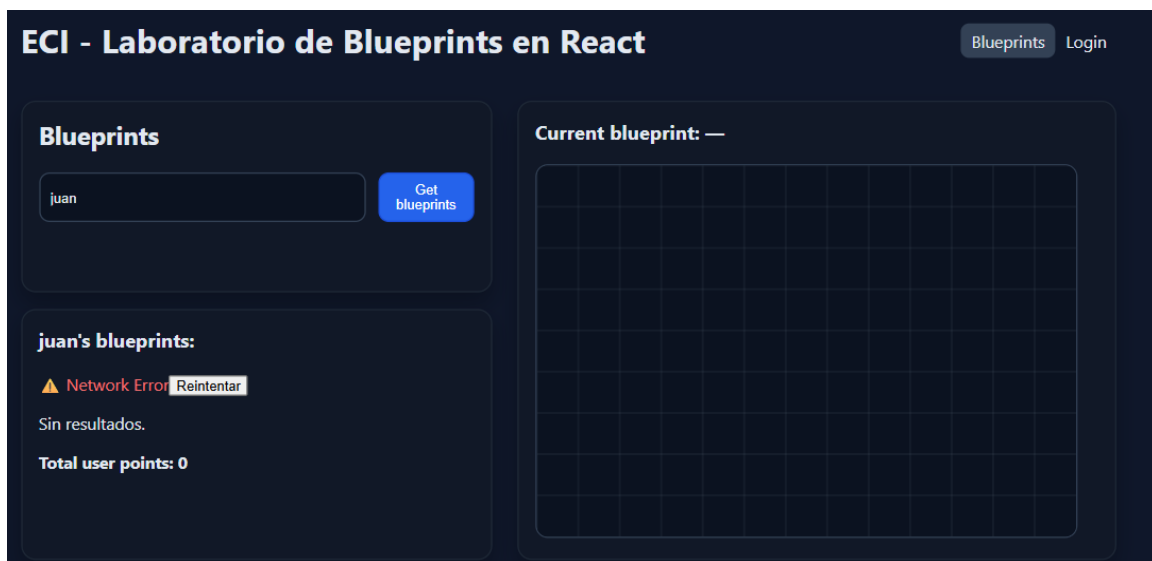
```
<div className="card">
  <h3 style={{ marginTop: 0 }}>
    {selectedAuthor ? `${selectedAuthor}'s blueprints:` : 'Results'}
  </h3>
  {status === 'loading' && <p>Cargando...</p>}
  {status === 'failed' && (
    <div style={{ color: '#f87171' }}>
      ⚠ {error}
      <button onClick={() => dispatch(fetchByAuthor(selectedAuthor))}>
        Reintentar
      </button>
    </div>
  )}
</div>
```

Demostración del flujo de error:

- La petición queda en estado de carga:



- Si falla, aparece el banner con el botón Reintentar:



6. Testing

Se implementaron 8 tests distribuidos en 4 archivos, cubriendo tanto los reducers puros del slice como la interacción con los componentes:

- blueprintsSlice.test.jsx reducers puros: inicialización, fetchByAuthor pending/fulfilled/rejected, removeBlueprintOptimistic
- BlueprintCanvas.test.jsx: Render del componente canvas con Testing Library
- BlueprintForm.test.jsx: Interacción con el formulario y verificación del envío
- BlueprintsPage.test.jsx: Dispatch del thunk al interactuar con la UI

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: { port: 5173 },
  test: {
    globals: true,
    environment: 'jsdom',
    setupFiles: './tests/setup.js',
  },
})
```

- Mock mejorado

```
BlueprintsPage.test.jsx > makeStore > slice
import { describe, it, expect, vi } from 'vitest'
import { render, screen, fireEvent } from '@testing-library/react'
import { Provider } from 'react-redux'
import { configureStore, createSlice } from '@reduxjs/toolkit'
import BlueprintsPage from '../src/pages/BlueprintsPage.jsx'

// Mock de thunks del slice para no requerir backend
vi.mock('../src/features/blueprints/blueprintsSlice.js', () => ({
  fetchAuthors: () => ({ type: 'blueprints/fetchAuthors' }),
  fetchByAuthor: (author) => ({ type: 'blueprints/fetchByAuthor', payload: author }),
  fetchBlueprint: (payload) => ({ type: 'blueprints/fetchBlueprint', payload }),
  deleteBlueprint: (payload) => ({ type: 'blueprints/deleteBlueprint', payload }),
  updateBlueprint: (payload) => ({ type: 'blueprints/updateBlueprint', payload }),
  selectTop5: () => [],
}))
```


- Implementación de los nuevos test

```
it('fetchByAuthor.fulfilled guarda items en byAuthor', () => {
  const items = [{ author: 'john', name: 'house', points: [] }]
  const action = { type: 'blueprints/fetchByAuthor/fulfilled', payload: { author: 'john', items } }
  const state = reducer(undefined, action)
  expect(state.byAuthor['john']).toEqual(items)
})

it('fetchByAuthor.pending pone status en loading', () => {
  const state = reducer(undefined, { type: 'blueprints/fetchByAuthor/pending' })
  expect(state.status).toBe('loading')
})

it('fetchByAuthor.rejected guarda el error', () => {
  const state = reducer(undefined, { type: 'blueprints/fetchByAuthor/rejected', error: { message: 'fallo' } })
  expect(state.status).toBe('failed')
  expect(state.error).toBe('fallo')
})

it('removeBlueprintOptimistic elimina del byAuthor', () => {
  const initial = {
    authors: [],
    byAuthor: { john: [{ author: 'john', name: 'house', points: [] }] },
    current: null,
    status: 'idle',
    error: null,
  }
  const state = reducer(initial, { type: 'blueprints/removeBlueprintOptimistic', payload: { author: 'john', name: 'house' } })
  expect(state.byAuthor['john']).toHaveLength(0)
})
```

- Confirmación de que las pruebas pasan

```
✓ tests/BlueprintsPage.test.jsx (1 test) 160ms

Test Files  4 passed (4)
   at commitPassiveMountEffects_complete (C:\Users\robin\Downloads\LAB06_ARSW\node_modules\react-dom\cjs\react-dom.development.js:24930:9)
   at commitPassiveMountEffects_begin (C:\Users\robin\Downloads\LAB06_ARSW\node_modules\react-dom\cjs\react-dom.development.js:24917:7)
   at commitPassiveMountEffects (C:\Users\robin\Downloads\LAB06_ARSW\node_modules\react-dom\cjs\react-dom.development.js:24905:3)
   at flushPassiveEffectsImpl (C:\Users\robin\Downloads\LAB06_ARSW\node_modules\react-dom\cjs\react-dom.development.js:27078:3) undefined

✓ tests/BlueprintsPage.test.jsx (1 test) 160ms

Test Files  4 passed (4)

✓ tests/BlueprintsPage.test.jsx (1 test) 160ms

Test Files  4 passed (4)
   Tests  8 passed (8)
Test Files  4 passed (4)
   Tests  8 passed (8)
Start at  00:01:18
   Tests  8 passed (8)
Start at  00:01:18
Start at  00:01:18
Duration  3.61s (transform 328ms, setup 825ms, collect 2.61s, tests 498ms, environment 5.63s, prepare 1.12s)

Duration  3.61s (transform 328ms, setup 825ms, collect 2.61s, tests 498ms, environment 5.63s, prepare 1.12s)
```

7. CI / Lint / Format

Se configuró un workflow de GitHub Actions en `.github/workflows/ci.yml` que ejecuta en cada push o pull request los tres pasos del pipeline: lint, test y build.

```
name: node-ci
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'
      - run: npm ci || npm install
      - run: npm run lint
      - run: npm test
      - run: npm run build
    env:
      VITE_API_BASE_URL: http://localhost:8080/api
      VITE_USE MOCK: 'true'
```

Lint

```
● PS C:\Users\robin\downloads\LAB06_ARSW> npm run lint

> blueprints-react-lab@0.1.0 lint
> eslint .

● PS C:\Users\robin\downloads\LAB06_ARSW> npm run build

> blueprints-react-lab@0.1.0 build
> vite build

vite v7.3.1 building client environment for production...
✓ 106 modules transformed.
dist/index.html      0.42 kB | gzip: 0.28 kB
dist/assets/index-Ckm_caCo.css 0.96 kB | gzip: 0.53 kB
dist/assets/index-C46LazxQ.js 243.83 kB | gzip: 82.93 kB
✓ built in 1.65s
❖ PS C:\Users\robin\downloads\LAB06_ARSW> 
```

```
Run npm run lint

1 ▶ Run npm run lint
4
5 > blueprints-react-lab@0.1.0 lint
6 > eslint .
7
```

Test

```
Run npm test 2s

79 at HTMLCanvasElement.mockCall (file:///home/runner/work/LAB06_ARSW/LAB06_ARSW/node_modules/@vitest/spy/dist/index.js:96:15)
80 at HTMLCanvasElement.getContext (file:///home/runner/work/LAB06_ARSW/LAB06_ARSW/node_modules/tinyspy/dist/index.js:47:80)
81 at /home/runner/work/LAB06_ARSW/LAB06_ARSW/src/components/BlueprintCanvas.jsx:10:24
82 at commitHookEffectListMount (/home/runner/work/LAB06_ARSW/LAB06_ARSW/node_modules/react-dom/cjs/react-dom.development.js:23189:26)
83 at commitPassiveMountOnFiber (/home/runner/work/LAB06_ARSW/LAB06_ARSW/node_modules/react-dom/cjs/react-dom.development.js:24970:11)
84 at commitPassiveMountEffects_complete (/home/runner/work/LAB06_ARSW/LAB06_ARSW/node_modules/react-dom/cjs/react-dom.development.js:24930:9)
85 at commitPassiveMountEffects_begin (/home/runner/work/LAB06_ARSW/LAB06_ARSW/node_modules/react-dom/cjs/react-dom.development.js:24917:7) undefined
86
87 ✓ tests/BlueprintCanvas.test.jsx (1 test) 49ms
88
89 Test Files 4 passed (4)
90 Tests 8 passed (8)
91 Start at 05:19:19
92 Duration 2.04s (transform 175ms, setup 263ms, collect 864ms, tests 287ms, environment 2.29s, prepare 399ms)
93
```

Build

```
Run npm run build 2s

1 ▶ Run npm run build
7
8 > blueprints-react-lab@0.1.0 build
9 > vite build
10
11 vite v7.3.1 building client environment for production...
12 transforming...
13 ✓ 106 modules transformed.
14 rendering chunks...
15 computing gzip size...
16 dist/index.html 0.41 kB | gzip: 0.27 kB
17 dist/assets/index-Ckm_caCo.css 0.96 kB | gzip: 0.53 kB
18 dist/assets/index-C46LazxQ.js 243.83 kB | gzip: 82.93 kB
19 ✓ built in 1.31s
```

8. Docker (opcional)

Se crearon los archivos Dockerfile y docker-compose.yml para contenerizar el frontend y el backend de forma conjunta. La estructura de carpetas utilizada es:

Downloads/

- └─ LAB05_ARSW/ Dockerfile del backend
- └─ LAB06_ARSW/ Dockerfile + docker-compose.yml

```
r-cd C:\Users\robin\downloads\LAB06_ARSW\Dockefile }
>Run All Services | docker-compose.yml - The Compose specification establish
services:
  >Run Service
  backend:
    build:
      context: ../LAB05_ARSW
      dockerfile: Dockerfile
    ports:
      - '8080:8080'
  >Run Service
  frontend:
    build: .
    ports:
      - '5173:4173'
    environment:
      - VITE_API_BASE_URL=http://backend:8080/api
      - VITE_USE MOCK=false
    depends_on:
      - backend
```

- Se ejecutó el build con docker-compose up --build, que construyó ambas imágenes y levantó los contenedores:

```
> [backend build 2/6] WORKDIR /app 4.1s
> [backend build 3/6] COPY pom.xml . 1.3s
> [backend build 4/6] RUN mvn -B -f pom.xml dependency:go-offline 170.8s
> [frontend build 3/6] COPY package*.json . 1.5s
> [frontend build 4/6] RUN npm ci || npm install 56.4s
> [backend build 5/6] COPY src ./src 2.7s
> [backend build 6/6] RUN mvn -B -DskipTests package 45.9s
> [frontend build 5/6] COPY . . 42.3s
> [frontend build 6/6] RUN npm run build 21.5s
> [backend stage-1 3/3] COPY --from=build /app/target/*.jar ./app.jar 1.6s
> [backend] exporting to image 9.5s
> => exporting layers 6.8s
> => exporting manifest sha256:a36c29a7afd3098e620ee618ccc4479510b0fd67793501cf5db0624c3b525ce1 0.0s
> => exporting config sha256:2f12460472fc5bae4738af8925f291ee352c0d70b9b0def6ddfe8b6f8173427a4 0.0s
> => exporting attestation manifest sha256:109daee6dccc01ala0a95d86ed0b6572e84510390a325bdbf9b0ee22e11b6359c 0.1s
> => exporting manifest list sha256:ccfd85a3d8b173148092378317071c28f52bb094cc9d979401661d921b9e78df 0.0s
> => naming to docker.io/library/lab06_arsw-backend:latest 0.0s
> => unpacking to docker.io/library/lab06_arsw-backend:latest 2.2s
> [backend] resolving provenance for metadata file 0.3s
> [frontend stage-1 4/4] COPY --from=build /app/dist ./dist 3.3s
> [frontend] exporting to image 12.9s
> => exporting layers 11.3s
> => exporting manifest sha256:f094509c8b9843381925fb5b688239541a22aaa1908e2fc5fd3e031104ec55e 0.0s
> => exporting config sha256:187d63e441de6599d36cee4d7f5c5a8d997d193d39647a62de8773604d5dcfd1 0.0s
> [frontend] exporting to image 12.9s
> => exporting layers 11.3s
> => exporting manifest sha256:f094509c8b9843381925fb5b688239541a22aaa1908e2fc5fd3e031104ec55e 0.0s
> => exporting config sha256:187d63e441de6599d36cee4d7f5c5a8d997d193d39647a62de8773604d5dcfd1 0.0s
> => exporting attestation manifest sha256:187d63e441de6599d36cee4d7f5c5a8d997d193d39647a62de8773604d5dcfd1 0.0s
> => exporting manifest list sha256:94af9204ac80ac11c569251630eda256c4f3c29a1bc8372122f9e3bfb396628 0.1s
> => exporting config sha256:4e2fd462bf747e330ea72cda8a82e445664f53de3e334dd108c95400d4a7a16 0.0s
> => naming to docker.io/library/lab06_arsw-frontend:latest 0.0s
> => unpacking to docker.io/library/lab06_arsw-frontend:latest 1.2s
> [frontend] resolving provenance for metadata file 0.1s
[+] Running 5/5
  ✓ lab06_arsw-backend Built 0.0s
  ✓ Network lab06_arsw_default Created 0.2s
  ✓ Container lab06_arsw-backend-1 Created 1.1s
  ✓ Container lab06_arsw-frontend-1 Created 0.2s
Attaching to backend-1, frontend-1
Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:8080 -> 127.0.0.1:0: listen tcp 0.0.0.0:8080: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.
```

- Tras liberar el puerto 8080 que tenía ocupado el backend local, se volvió a ejecutar docker-compose up y ambos servicios arrancaron correctamente:

```
PS C:\Users\robin\downloads\LAB06_AWSM> docker-compose up
Attaching to backend-1, frontend-1
Frontend-1 | INFO Accepting connections at http://localhost:4173
backend-1 |
backend-1 | _____
backend-1 | \o/ Spring Boot \o/
backend-1 | _____
backend-1 | :: Spring Boot ::      (v3.3.2)
backend-1 |
backend-1 | 2026-03-12T05:22:57.966Z INFO I --- [main] c.e.e.b.BlueprintsApiApplication : Starting BlueprintsApiApplication v2.0.0 using Java 21.0.10 with PID 1 (/app/app.jar started by root in /app)
backend-1 | 2026-03-12T05:22:57.971Z INFO I --- [main] o.s.b.w.embedded.Tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
backend-1 | 2026-03-12T05:23:00.496Z INFO I --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
backend-1 | 2026-03-12T05:23:00.518Z INFO I --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.26]
backend-1 | 2026-03-12T05:23:00.519Z INFO I --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
backend-1 | 2026-03-12T05:23:00.602Z INFO I --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2476 ms
backend-1 | 2026-03-12T05:23:02.733Z INFO I --- [main] o.s.b.w.embedded.Tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
backend-1 | 2026-03-12T05:23:02.767Z INFO I --- [main] c.e.e.b.BlueprintsApiApplication : Started BlueprintsApiApplication in 6.58 seconds (process running for 8.396)
```

- frontend-1 corriendo en http://localhost:4173 SI
- backend-1 Spring Boot en puerto 8080 SI
- BlueprintsApiApplication iniciada en 6.58 segundos SI